

PROJECT REPORT OF CO-OP Project at Industry (Module-2)
ON
Monolithic Vs Microservices Architecture (Research Paper)
submitted in partial fulfilment of the requirements for the award of degree of
BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Aditya Bajaj (2210991185)

Ayushmaan (2210990207)

Amandeep Singh(2210991250)

Supervised By:

Dr. Monika Aggarwal

Assistant Professor

Chitkara University



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

DECLARATION

I hereby certify that the work which is being presented in the project report entitled "Comparative Analysis of Microservices and Monolithic Architectures" in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of our own work carried out under the supervision of our mentor. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

(Ayushmaan)

(Aditya Bajaj)

Date: 18th May 2026

(Amandeep Singh)

This is to certify that the above statement made by the candidates is correct to the best of my knowledge and belief.

Dr. Monika Aggarwal

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

CONTENTS

S.No.	Title	Page No.
	Declaration	2
	Acknowledgement	4
	List of Figures and Tables	5
	Abstract	6
1	Introduction	7
2	Literature Review	9
3	Architecture Fundamentals	10
4	Methodology	11
5	Comparative Analysis & Performance Evaluation	13
6	Case Studies and Migration Patterns	15
7	Discussion	16
8	Conclusion and Future Scope	17
	References	18

ACKNOWLEDGEMENT

The authors would like to express sincere gratitude to the Department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology for their invaluable guidance, mentorship, and computational resources that greatly aided the experiments and research conducted for this project.

We extend our thanks to our supervisor and mentor for providing continuous direction and feedback throughout the course of this project. Their insights and constructive criticism have been instrumental in shaping the quality of this report.

We also acknowledge the academic and industry researchers whose published works formed the backbone of our literature review, and the open-source communities behind Spring Boot, Apache JMeter, Prometheus, and Grafana, whose tools were central to our benchmarking methodology.

Finally, we are grateful to our families and peers for their constant encouragement and support during the entire duration of this co-op project.

LIST OF FIGURES AND TABLES

Tables

Table No.	Title	Page No.
1	Table 1: Architectural Performance and Complexity Comparison	5
2	Table 2: Throughput Capabilities and Response Thresholds	6
3	Table 3: Documented Microservices Reversions	7
4	Table 4: Architectural Selection Decision Matrix	8

ABSTRACT

The software architecture design choice is a critical decision that determines the performance, scalability, and development speed of an application. Traditionally, monolithic architecture has been the preferred choice, where components are tightly coupled within a single deployment unit. However, with the advent of microservices, the landscape has shifted toward independent services loosely coupled via APIs.

This study compares microservices and monolithic architectures across key dimensions: scalability, maintainability, deployment complexity, operational overhead, and cost. Empirical performance tests were conducted at varying load levels (1,000 to 200,000 concurrent users) using Apache JMeter and Gatling on a Spring Boot-based reference application hosted on Microsoft Azure. Architectural complexity was assessed using Legacy Complexity Index (LCI) and Cognitive Complexity Points (CCP). Real-world case studies from organizations including Swiggy, Razorpay, and Amazon Prime Video were also analyzed.

Results show microservices outperform monoliths at scale, scoring 4.7/5 in scalability versus 2.1/5 for monoliths. Monoliths, however, excel in operational simplicity (4.3/5) and exhibit 2-3x faster response times under low load. The architectural crossover point is identified at approximately 25 independent services. A decision framework is proposed to help software architects select the appropriate architecture based on team size, traffic patterns, and DevOps maturity.

Keywords: Microservices, Monolithic Architecture, Scalability, Software Architecture, System Performance, Deployment Complexity.

1. Introduction

The architectural paradigm that defines contemporary software engineering is a constant search for balance between system performance, operational manageability, and team agility. For many decades, the monolithic paradigm formed the foundation of application design and development, defined by the consolidation of business logic, user interfaces, and data access layers into a cohesive unit of deployment.

However, in recent years, cloud computing, the proliferation of mobile internet access, and the need for continuous delivery have revealed the structural inflexibility inherent in large-scale monolithic systems. As a direct result of this evolution, microservices architecture emerged as an alternative that breaks down a monolithic application into a set of autonomous services that interact through well-defined APIs.

By 2026, cloud-native design has become so prevalent that nearly 85% of enterprise workloads are containerized or orchestrated. The research addresses a core problem: the quantification of trade-offs between these two architectural styles. While microservices are generally considered more scalable and fault-tolerant, they carry a "microservices premium" in terms of operational and network overhead. On the other hand, monolithic systems offer nanosecond-level internal latency and are significantly easier to debug.

1.1 Problem Statement

Software architects face the challenge of identifying the "crossover point" — at what system size and team headcount the complexity of a monolithic architecture outweighs that of a microservices mesh. This research empirically quantifies these trade-offs to support informed architectural decisions.

1.2 Objectives

- Develop a comparative matrix to measure technical and organizational performance of both architectures.
- Employ the Universal Scalability Law (USL) to determine scalability characteristics and predict upper limits of throughput.

- Examine patterns of successful and failed architecture migrations to identify best practices for 2024–2026.
- Provide a decision framework for software architects based on team structure, traffic patterns, and DevOps maturity.

1.3 Scope of the Study

This study covers both technical and organizational dimensions of architectural selection. Performance benchmarking was conducted at loads ranging from 1,000 to 200,000 concurrent users. Real-world case studies from Indian startups (Swiggy, Razorpay) and global enterprises (Amazon Prime Video, Segment) are analyzed to complement the empirical findings.

2. Literature Review

Academic and industrial research in software architecture has evolved from static evaluation of structural patterns to dynamic assessment of system evolution and operational robustness. The monolithic architecture paradigm, historically dominant, was optimized for single-machine resource efficiency in the late 20th century.

2.1 The Genesis of Microservices and Early Successes

The codification of microservices in 2011–2012 emerged as a response to the "Software Crisis 2.0," where monolithic enterprise systems had become too large for any single programmer to comprehend. Netflix and Amazon were early adopters, demonstrating the advantages of service decomposition for supporting thousands of daily releases. Martin Fowler and Sam Newman's research established foundational principles, emphasizing decoupled databases and bounded contexts. Between 2015 and 2020, the literature was overwhelmingly positive about microservices' promises of infinite scalability and technology flexibility.

2.2 The Realist Turn and Re-Monolithization (2022–2026)

Recent peer-reviewed research has introduced necessary skepticism. Blinowski et al. (2022) demonstrated through controlled experiments that monolithic architectures outperform microservices in CPU-intensive tasks on single machines due to the absence of network serialization overhead. A notable trend of "re-monolithization" has emerged, with organizations like Amazon Prime Video and Segment publicly disclosing their decision to revert from microservices to monolithic or modular monolith architectures — in Amazon's case reducing infrastructure costs by over 90%.

2.3 Emerging Metrics and Mathematical Models

To move beyond anecdotal evidence, researchers have applied the Universal Scalability Law (USL) to software systems, enabling prediction of throughput plateaus due to contention and coherency costs. The development of Cognitive Complexity Points (CCP) has further quantified the "maintainability debt" of complex service chains. In 2026, AI-augmented architecture tooling is emerging as a potential mitigation for microservices complexity.

3. Architecture Fundamentals

3.1 Monolithic Architecture: Principles and Constraints

A monolithic application is architected as a single logical executable containing all business capabilities. At runtime, all components coexist in a shared memory space. The guiding principle is "unified deployment" — even a minor change to one module requires rebuilding and redeploying the entire application. Communication between modules occurs via in-process method calls executing at CPU cache speed (nanoseconds), making monoliths ideal for performance-critical applications.

The primary constraint is vertical scaling — more load demands more powerful hardware. Horizontal scaling requires deploying the entire application across multiple servers, even if only one function needs scaling. Additionally, a centralized single database creates a bottleneck and single point of failure, as physical connection limits are eventually reached.

3.2 Microservices Architecture: Decentralization and Distribution

Microservices implement decentralized governance by splitting an application into independently deployable services, each with its own private data storage. Core principles include:

- Bounded Contexts: Each service is built around a specific business domain context, preventing cross-service propagation of changes.
- Loose Coupling: Services communicate via APIs, enabling independent evolution.
- Polyglot Persistence: Each service can use the most suitable database technology (e.g., graph DB for social connections, document DB for product catalogs).
- API Gateway and Service Mesh: A central API gateway handles routing; a service mesh (e.g., Istio) manages service-to-service communication.

The primary technical challenge is network-induced latency. Every inter-service call (HTTP/REST or gRPC) introduces 10–100ms of latency, orders of magnitude higher than an in-memory call. Asynchronous communication via message brokers (Kafka, RabbitMQ) is typically employed to mitigate this, but introduces additional complexity around eventual consistency and distributed transactions.

4. Methodology

This study employs a multi-methodological approach combining theoretical architectural analysis with empirical performance testing to provide both qualitative and quantitative insights into the trade-offs between the two architectures.

4.1 Systematic Literature Review

A systematic review of over 60 peer-reviewed papers, technical reports, and industry whitepapers published between 2020 and 2026 was conducted. Emphasis was given to multivocal literature covering both academic research and industry reports from Netflix, Amazon, and Segment. This review identified the primary evaluation metrics used in the study.

4.2 Performance Benchmarking Framework

A reference sales management web application was implemented in two architectural styles using the Java Spring Boot framework:

- **Monolithic Testbed:** All application logic (authentication, inventory, warehouse, order processing) packaged as a single .war file, deployed on Tomcat with a single PostgreSQL database.
- **Microservices Testbed:** Application decomposed into five independent microservices with Eureka for service discovery, Spring Cloud Gateway for API routing, and a dedicated PostgreSQL database per service.

4.3 Testing Environment and Tools

- **Load Testing:** Apache JMeter and Gatling for simulating concurrent user requests.
- **Performance Monitoring:** Prometheus and Grafana for real-time resource monitoring.
- **Cloud Environment:** Microsoft Azure App Service for cloud-native, real-world simulation.

4.4 Variable Load Testing

Variable load testing was conducted at four load stages: 1,000 / 10,000 / 100,000 / 200,000 concurrent users. Each stage was run multiple times to ensure consistency and reduce performance variability.

4.5 Complexity Metric Calculation

Two complexity metrics were applied to both experimental implementations and real-world case study systems (Swiggy, Razorpay, Amazon Prime Video):

- Legacy Complexity Index (LCI): Quantifies structural complexity of the codebase.
- Cognitive Complexity Points (CCP): Measures maintainability debt from interdependent service chains.

5. Comparative Analysis and Performance Evaluation

5.1 Technical Metric Comparison

The table below summarizes architectural performance across key technical dimensions:

Metric	Monolithic	Microservices	Observation
Scalability Score	2.1 / 5.0	4.7 / 5.0	Microservices scale more efficiently
Operational Complexity	4.3 / 5.0	2.8 / 5.0	Monolith easier to manage
Network Latency	Low	Higher (2–3x)	MS adds network overhead
Deployment Complexity	4.8 / 5.0	2.2 / 5.0	Monolith simpler to deploy
Resource Utilization	2.4 / 5.0	4.6 / 5.0	MS allows granular scaling
Fault Tolerance	1.8 / 5.0	4.8 / 5.0	MS isolates failures effectively

Table 1: Architectural Performance and Complexity Comparison

5.2 Response Time and Throughput Analysis

Under low load (1,000 concurrent users), the monolithic system achieved a median response time of 8ms, while the microservices system had an average of 24ms due to API Gateway routing and network overhead. However, at higher loads, microservices architecture maintained performance by dynamically scaling containers:

Concurrent Users	Monolith Throughput (req/s)	Microservices Throughput (req/s)	Failure Rate
1,000	1,000	1,000	0.00%
10,000	8,500	9,200	0.02%
100,000	45,000 (Limit)	88,000	2.10% (MS) vs 12% (ML)
200,000	System Crash	165,000	4.80% (MS)

Table 2: Throughput Capabilities and Response Thresholds

5.3 Resource Utilization and Elasticity

Monitoring via Prometheus and Grafana revealed that microservices architecture provides more granular CPU utilization. When the Order Service experienced high request volume, only that service scaled up, leaving other services unaffected. In the monolithic architecture, all modules experienced increased CPU load regardless of which function was under stress. This demonstrates the inherent resource efficiency advantage of microservices at scale.

5.4 Architectural Crossover Point

The empirical crossover point was identified at approximately 25 independent services. Below this threshold, microservices introduce unnecessary complexity with no substantial performance advantage. Beyond this threshold, a monolithic architecture becomes increasingly difficult to maintain and scale, making microservices the more efficient choice.

6. Case Studies and Migration Patterns

6.1 Successful Hyper-Growth Migrations

Swiggy and Zomato initially developed monolithic architectures before transitioning to microservices to support hyper-growth. Swiggy's decomposition enabled regional feature rollouts and live A/B experiments without disrupting the primary consumer application. Razorpay adopted the Strangler Fig Pattern — incrementally replacing monolithic components with microservices using dual-write approaches to maintain data consistency during the transition.

6.2 Architectural Reversion

Several prominent organizations have publicly reverted from microservices to monolithic or modular monolith architectures, demonstrating that microservices are not universally optimal:

Organization	Component	Reason for Reversion	Outcome
Amazon Prime Video	Monitoring Service	High infrastructure cost	90% cost reduction
Segment	Event Routing	Operational complexity	10x throughput increase
Istio	Control Plane	Management overhead	Simplified deployment
InVision	Legacy Core	Small team size	Improved dev focus

Table 3: Documented Microservices Reversions

7. Discussion

7.1 Infrastructure Cost and FinOps Considerations

In modern cloud environments, infrastructure cost is a critical architectural criterion. Although microservices offer improved resource allocation and fine-grained scaling, they require additional infrastructure components including container orchestration tools, service meshes, and observability platforms. Studies suggest microservices systems may incur infrastructure costs 2–4 times higher than equivalent monolithic systems, particularly at lower scales where the scaling benefit does not offset the overhead.

7.2 Organizational Maturity and Conway's Law

Conway's Law states that the structure of a system mirrors the communication structure of the organization that built it. This research identifies three organizational maturity phases:

- Phase I (< 15 developers): Monolithic architecture is generally preferable due to lower operational overhead and simpler deployment.
- Phase II (15–50 developers): Organizations experience increasing architectural pressure as the monolith becomes harder to maintain and teams begin stepping on each other.
- Phase III (> 50 developers): Microservices architectures enable team autonomy, parallel development, and independent deployments.

7.3 Architectural Selection Framework

Based on findings, the following decision matrix is proposed:

Criteria	Choose Monolith If	Choose Microservices If
Team Structure	Small centralized team	Large distributed teams
Traffic Pattern	Predictable workloads	Highly variable workloads
Scaling Strategy	Vertical scaling sufficient	Horizontal scaling required
Data Consistency	Strong ACID consistency needed	Eventual consistency acceptable
Deployment Frequency	Monthly releases	Continuous deployment
DevOps Capability	Limited automation	Advanced CI/CD infrastructure

Table 4: Architectural Selection Decision Matrix

8. Conclusion and Future Scope

8.1 Conclusion

This study demonstrates that neither microservices nor monolithic architecture is universally superior. The optimal choice depends on a combination of technical, organizational, and operational factors. Key findings include:

- Microservices achieve a scalability score of 4.7/5 compared to 2.1/5 for monoliths, and support up to 200,000 concurrent users before degradation.
- Monolithic architectures offer 2–3x faster response times under low load and simpler operational management (4.3/5 vs. 2.8/5).
- The architectural crossover point is approximately 25 independent services, beyond which microservices become more cost-effective.
- Conway's Law is a reliable predictor of architectural success — organizational structure must align with technical structure for either approach to succeed.
- Infrastructure costs for microservices can be 2–4x higher, and some organizations have successfully reverted to monolithic designs to reduce costs and complexity.

The decision to adopt microservices should be driven by validated scale predictions, clear DevOps maturity, and distributed team structures — not by industry trends alone.

8.2 Future Scope

Future research directions include:

- Hybrid architectures: Exploring modular monolith patterns that combine the simplicity of monoliths with selective microservice decomposition for critical domains.
- AI-augmented microservices management: Investigating how AI copilots can automate service mesh configuration, inter-service contract validation, and glue code generation to reduce operational complexity.
- Serverless integration: Evaluating the role of FaaS (Function-as-a-Service) platforms in bridging the gap between monolithic simplicity and microservices scalability.
- FinOps optimization models: Developing predictive cost models that help teams calculate the financial break-even point for microservices adoption based on traffic projections.

References

- [1] Baddula, P.: The Evolution of Software Architecture: Monolithic to Microservices. Medium. Accessed: 14-Mar-2026 (2023).
- [2] Orkes: Evolution of Software Architecture: From Mainframes and Monoliths to Distributed Computing. orkes.io. Accessed: 14-Mar-2026 (2024).
- [3] Leapcell: The Evolution of Architecture: Monolithic vs Microservices. dev.to. Accessed: 14-Mar-2026 (2023).
- [4] Jones, S.: From Monoliths to Microservices and Beyond. Better Programming (2021).
- [5] IBM Cloud Education: Evolution of Microservices. developer.ibm.com. Accessed: 14-Mar-2026 (2021).
- [6] Dragoni, N., et al.: Microservices: Yesterday, today, and tomorrow. In: Present and Ulterior Software Engineering, pp. 195–216. Springer, Cham (2017).
- [7] Taibi, D., Lenarduzzi, V., Pahl, C.: Microservices anti-patterns: A taxonomy. In: Microservices: Science and Engineering. Springer (2020).
- [8] Fowler, M.: Microservices Guide. martinowler.com. Accessed: 14-Mar-2026 (2019).
- [9] DZone: Evolution of Software Architecture: From Monoliths to Microservices. DZone Architecture (2022).
- [10] Richardson, C.: Microservices Patterns: With Examples in Java. Manning Publications (2018).
- [11] Fowler, M., Lewis, J.: Microservices. martinowler.com. Accessed: 14-Mar-2026 (2014).
- [12] Blinowski, G., et al.: Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation. IEEE Access (2022).